



What is GitHub Actions?

Automate CI/CD Workflows Directly from GitHub



WHAT IS GITHUB ACTIONS?

GitHub Actions is GitHub's built-in CI/CD platform that automates software development workflows. It automatically builds, tests, packages, and deploys your application whenever a specific event occurs, such as a code push or pull request.

KEY FEATURES

- ✓ Native GitHub CI/CD
- ✓ Event-Driven Automation
- ✓ Build, Test & Deploy
- ✓ GitHub Hosted & Self-Hosted Runners
- ✓ Supports Docker & Secret Management
- ✓ Thousands of Marketplace Actions
- ✓ Thousands of Multi-Platform Support

HOW GITHUB ACTIONS WORKS



REAL PRODUCTION EXAMPLE

push pull_request schedule release
pull_request workflow_dispatch workflow_call

PRODUCTION BENEFITS

- ✓ Faster Software Delivery
- ✓ Continuous Deployment
- ✓ Automated Testing
- ✓ Reduced Human Errors
- ✓ Continuous Integration
- ✓ Faster Developer Feedback

INTERVIEW QUESTIONS

- What is GitHub Actions?
- Why is GitHub Actions used?
- What is a GitHub Runner?
- Which events trigger a workflow?
- How is GitHub Actions different from Jenkins?

INTERVIEW TIP

GitHub Actions is an event-driven automation platform. Every workflow starts when a configured event (such as push, pull request, or schedule) occurs.



Code Push



GitHub Actions



Build



Test



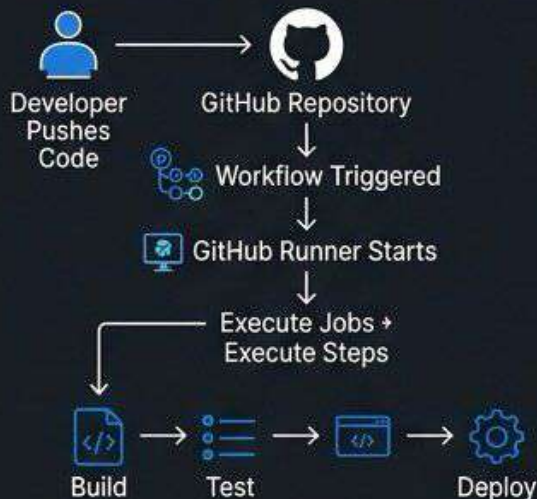
Deploy

Automate your entire **CI/CD pipeline** directly from GitHub.

How GitHub Actions Works

Workflow, Events, Jobs, Steps & Runners Explained

WORKFLOW EXECUTION



KEY COMPONENTS

Workflow

Defines a workflow in a YAML file stored in `.github/workflows/`

Event

Triggers such as:

- push
- pull_request
- workflow_dispatch
- schedule

Runner

Virtual machines such as ubuntu-machines: ubuntu-latest, windows-latest, macos-latest

Job

Groups group related tasks like, such as a:

- Build
- Test
- Deploy

Step

Individual tasks within a job like were:

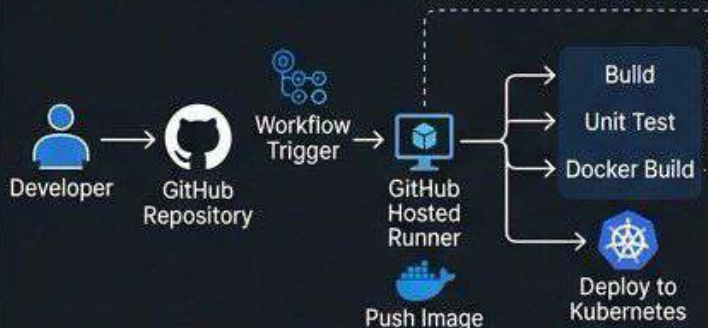
- Checkout Code
- Install Dependencies
- Run Tests
- Build Docker Image

Action

Reusable automation from GitHub Marketplace examples:

- actions/checkout
- actions/setup-node
- docker/login-action

PRODUCTION ARCHITECTURE



REAL PRODUCTION EXAMPLE



BEST PRACTICES

- ✓ Keep Build, Test, and Deploy separate jobs
- ✓ Use reusable Actions from GitHub Marketplace
- ✓ Store credentials in GitHub Secrets
- ✓ Cache dependencies for faster builds
- ✓ Use descriptive workflow names
- ✓ Monitor failed workflow executions

INTERVIEW QUESTIONS

- What is a Workflow?
- What is the difference between a Job and a Step?
- What is a GitHub Runner?
- What is an Action?
- Which events can trigger a workflow?

INTERVIEW TIP

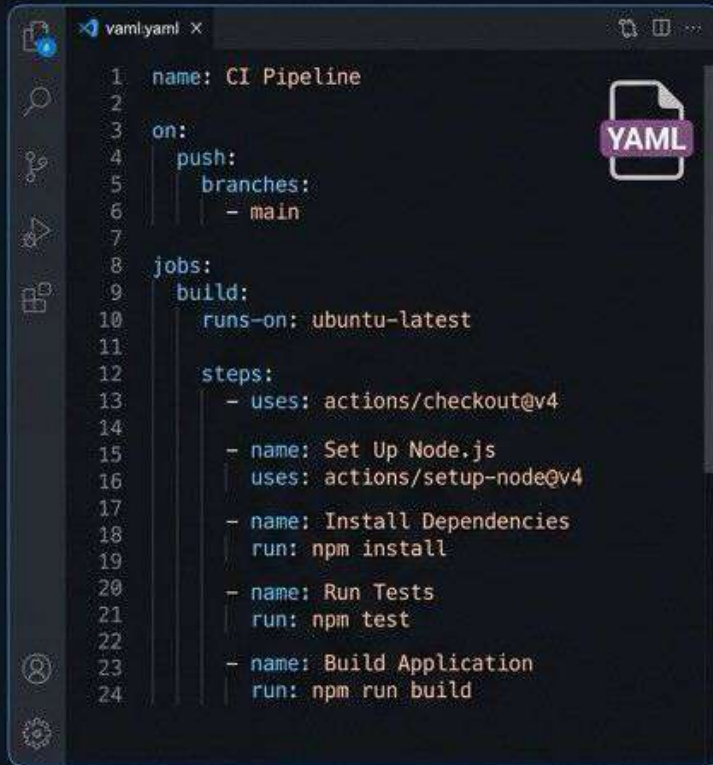
This is the execution hierarchy:

Event → **Workflow** → **Runner** → **Job** → **Step** → **Action**

This flow is frequently asked in GitHub Actions interviews.

Event → **Workflow** → **Runner** → **Job** → **Step** → **Build** → **Test** → **Deploy**

Understanding the **GitHub Actions** workflow execution is the key to mastering CI/CD.



- **name:** Defines the workflow name displayed in GitHub Actions.
- **on:** Defines the event that triggers the workflow.
 - Examples: push, pull_request, workflow_dispatch
- **branches:** Specifies which branch starts the workflow.
 - Example: main
- **jobs:** A collection of one or more jobs executed by the workflow.
- **build:** The name of the job.
 - Examples: build, test, deploy
- **runs-on:** Defines the operating system used by the GitHub Runner.
 - Examples: ubuntu-latest, windows-latest, macos-latest
- **steps:** A sequence of tasks executed one after another.
- **uses:** Runs a reusable GitHub Action from the GitHub Marketplace.
 - Examples: actions/checkout@v4, actions/setup-node@v4
- **run:** Executes shell commands on the GitHub Runner.
 - Examples: npm install, npm test, docker build, terraform apply

```

graph LR
    A[Developer Pushes Code] --> B[Workflow Triggered]
    B --> C[GitHub Runner Starts]
    C --> D[Checkout Repository]
    D --> E[Install Dependencies]
    E --> F[Run Tests]
    F --> G[Build Application]
    G --> H[Workflow Completed]
  
```

A developer pushes code to the main branch.
GitHub Actions automatically:

- ✓ Downloads the source code
- ✓ Sets up Node.js
- ✓ Installs dependencies
- ✓ Runs unit tests
- ✓ Builds the application
- ✓ Reports build status back to GitHub

- ✔ Keep workflows modular
- ✔ Use reusable Actions
- ✔ Store credentials in GitHub Secrets
- ✔ Store credentials in GitHub Secrets
- ✔ Separate Build, Test & Deploy jobs
- ✔ Use meaningful workflow names
- ✔ Validate YAML before merging

- What is the purpose of the **on** keyword?
- What is the difference between **uses** and **run**?
- Why is **runs-on** required?
- Where are GitHub Actions workflow files stored?
- What is a GitHub Runner?

Remember these seven keywords:
name → **on** → **jobs** → **runs-on** → **steps** → **uses** → **run**
 If you understand these keywords, you can read and write almost any GitHub Actions workflow.

Workflow YAML → Runner → Build → Test → Deploy

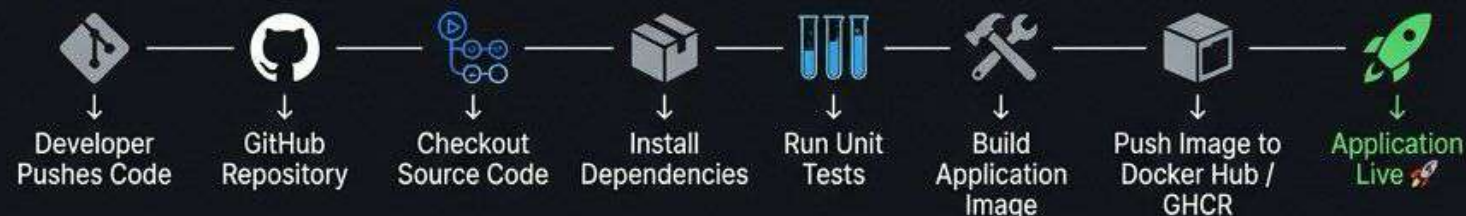
A well-structured YAML file is the foundation of every GitHub Actions CI/CD pipeline.

GitHub Actions in Production

Automating Build, Test & Deployment Pipelines



CI/CD PIPELINE



PRODUCTION WORKFLOW



REAL PRODUCTION EXAMPLE

A developer pushes code to the main branch.

GitHub Actions automatically:

- ✓ Pulls the latest source code
- ✓ Installs project dependencies
- ✓ Runs unit tests
- ✓ Builds the application
- ✓ Creates a Docker image
- ✓ Pushes the image to GitHub Container Registry
- ✓ Deploys the latest version to a Kubernetes cluster



Deployment is completed automatically without manual intervention.

PRODUCTION BENEFITS

- ✓ Faster CI/CD
- ✓ Automated Testing
- ✓ Zero Manual Deployment
- ✓ Consistent Releases
- ✓ Faster Feedback
- ✓ Faster Feedback
- ✓ Reduced Human Errors
- ✓ Easy Rollbacks
- ✓ Improved Developer Productivity

INTERVIEW QUESTIONS

- How does GitHub Actions deploy applications?
- Where should secrets be stored?
- Why separate Build, Test and Deploy jobs?
- How do GitHub Actions integrate with Docker and Kubernetes?
- How do you deploy only after successful tests?

BEST PRACTICES

- ✓ Keep Build, Test & Deploy as separate jobs
- ✓ Store credentials in GitHub Secrets
- ✓ Use reusable workflows
- ✓ Cache dependencies for faster builds
- ✓ Deploy only after successful tests
- ✓ Protect production branches



INTERVIEW TIP

A production GitHub Actions pipeline should always follow this order:

Checkout → Build → Test → Package → Deploy

Never deploy an application before all validation steps pass.

Git Push → Build → Test → Docker → Kubernetes → Production

Modern DevOps teams automate the entire software delivery lifecycle using GitHub Actions.



Learn DevOps Together
@LearnDevOpsTogether



GitHub Actions Troubleshooting

Common Issues, Best Practices & Interview Tips



COMMON ISSUES

- ❌ **Workflow Not Triggering**
causes:
 - Wrong trigger event
 - Incorrect branch name
 - Workflow file not in .github/workflows/
- ❌ **Build Failed**
causes: Syntax errors
 - Missing dependencies
 - Test failures
- ❌ **Permission Denied**
causes: Missing workflow permissions
 - Invalid GitHub Token
- ❌ **Secrets Not Found**
causes: Secret not created
 - Incorrect secret name
 - Wrong repository settings
- ❌ **Docker Build Failed**
causes: Invalid Dockerfile
 - Authentication failure
 - Missing image tag
- ❌ **Deployment Failed**
causes: Kubernetes credentials missing
 - Cluster unavailable
 - Invalid deployment manifest

TROUBLESHOOTING STEPS

- 1 Check Workflow Logs 
- 2 Verify Trigger Event 
- 3 Validate YAML Syntax 
- 4 Check GitHub Secrets 
- 5 Verify Runner Status 
- 6 Review Build Logs 
- 7 Redeploy After Fix 

BEST PRACTICES

- ✅ Store credentials in GitHub Secrets
- ✅ Validate YAML before committing
- ✅ Keep Build, Test & Deploy in separate jobs
- ✅ Enable branch protection rules
- ✅ Use reusable workflows
- ✅ Cache dependencies to speed up builds
- ✅ Keep Actions updated to the latest versions
- ✅ Monitor workflow execution regularly

REAL PRODUCTION SCENARIO

A developer pushes code to the main branch, but the deployment doesn't start.

Troubleshooting Process:

- ✅ Check Actions tab for failed workflow
- ✅ Verify the push event trigger
- ✅ Review workflow logs
- ✅ Validate GitHub Secrets
- ✅ Fix the YAML error
- ✅ Re-run the workflow

Deployment completes successfully 

INTERVIEW QUESTIONS

- Why is my GitHub Actions workflow not triggering?
- How do you debug a failed GitHub Actions pipeline?
- What are GitHub Secrets?
- What is the difference between GitHub-hosted and Self-hosted Runners?
- How do you secure a production workflow?



INTERVIEW TIP

Troubleshoot: Trigger → YAML → Runner → Secrets → Logs → Deployment

A structured troubleshooting approach is highly valued in DevOps interviews.

BOTTOM HIGHLIGHT

Trigger → Workflow → Logs → Fix → Re-run → Success

Master troubleshooting to build reliable production CI/CD pipelines



Learn DevOps Together @LearnDevOpsTogether

